SYSTEMS ANALYSIS & DESIGN | PHASE 2

EchoMessenger

Product, Requirements Fulfilment, and Design Conformance Report

Course	Systems Analysis and Design
Institution	Sharif University of Technology
Semester	Spring 2026
Status	Final Phase 2 submission
Repository	github.com/Sina-rokna/SD-EchoMessenger

Team
Sina Mohammadi • Mohammad Ermia Ghaseri • Mehrshad Valizadeh Arjmand
Amir Mohammad Shahrezaei • Amir Hossein Ghasemipour • Nima Notghi

Submission scope
All mandatory requirements, realtime chat and notifications, scheduled messages, automated tests, and a reproducible Docker runtime.

Contents

1. Executive Summary

Phase 2 turns the Phase 1 analysis package into a complete web messaging product. EchoMessenger supports secure account creation and session authentication, editable user profiles, direct chats, private groups, channels with topics, data-driven roles, text and media messages, message edit/delete, per-chat search, and persistent notifications.

Both bonus requirements are integrated into the normal product flow. Messages and notifications arrive through authenticated WebSockets without a manual refresh. A user can also schedule a message for a future time; the message is stored durably and delivered by Celery even when that user is offline.

The final architecture deliberately remains a modular monolith with separate runtime processes. This preserves the Phase 1 service-oriented and containerized design while avoiding the operational overhead of unnecessary microservices.

2. Phase 1 Continuity

2.1 Concepts preserved

The implemented domain retains the Phase 1 terminology and relationships: `User`, `ChatSpace`, `Topic`, `Role`, `User_ChatSpace` (implemented as `SpaceMembership`), and `Message`.

The three-column client follows the Phase 1 chat wireframe:

- left rail for direct chats, groups, and channels;
- centre area for the active topic, search, history, scheduling, and composer;
- right rail for members and their roles;
- modal profile view/edit flow.

2.2 Necessary schema extensions

The original ERD could not represent several explicit requirements. The final schema therefore adds:

- `DIRECT` as a ChatSpace type for user-to-user messages;
- owner and missing foreign keys for enforceable relationships;
- profile avatar, biography, and group-invite policy;
- complete role capability fields instead of one delete flag;
- message status, topic link, created/sent/edited/scheduled timestamps;
- `Attachment` for image, video, audio, and general files;
- `Notification` for persistent and live notification delivery.

These changes extend the Phase 1 model rather than replacing it. The detailed rationale is in `docs/phase2/architecture.md`.

2.3 Deliberate scope exclusions

SSO and voice-channel controls appeared only as generic placeholders in early wireframes. They were absent from the stated requirements, user stories, ERD, and selected technology flow, so they are not presented as fake or unfinished features in the final product. Reactions, calls, read receipts, and friend requests are also outside the course scope.

3. Final Architecture

The browser reaches one Nginx entry point. Nginx serves the React application, proxies `/api/` to Django, and upgrades `/ws/` connections to the same ASGI service. PostgreSQL is the durable source of truth. Redis is used by Django Channels for realtime fan-out. RabbitMQ carries Celery tasks, and Celery Beat initiates restart-safe scheduled-message scans.

Runtime services:

1. **Nginx gateway** - static client, HTTP routing, WebSocket upgrade, request limits, and security headers.
2. **Django ASGI** - REST API, session authentication, domain services, file authorization, and WebSocket consumers.
3. **PostgreSQL** - accounts, spaces, memberships, roles, topics, messages, attachments, and notifications.
4. **Redis** - low-latency Channels group delivery.
5. **RabbitMQ** - durable Celery broker.
6. **Celery worker and beat** - permission-aware scheduled delivery.

All services are reproducibly defined in `compose.yaml`. Local tests can use SQLite and an in-memory channel layer without changing product code.

4. Requirements Fulfilment

FR-01 - Registration, login, and logout

Users register with a unique username, unique email address, and validated password. Django hashes passwords and creates an HttpOnly session. Login uses email and a generic invalid-credential response. Registration and login are CSRF-protected, and logout invalidates the session.

FR-02 - Send and receive messages

Members send text, one or more attachments, or both. Direct chats, groups, and channel topics share the same validated message service. Sent history is ordered chronologically and paginated.

FR-03 - Edit and delete messages

Only the sender can edit sent text, and the response exposes an edited timestamp/tag. The sender and users with the specified management authority can delete. Every operation is checked again on the server, even if a client manually constructs the request.

FR-04 - Channels and topics

A channel creator becomes owner and therefore holds all capabilities. Authorized roles can create, rename, and delete topics. Every channel message must reference a topic belonging to that channel.

FR-05 - Groups

Users create a group with one or more other users. A target whose invite policy is `NOBODY` cannot be added by another user. Members can add users, leave the group, and perform the group-management actions required by the literal project statement.

FR-06 - Edit/delete channels and groups

A channel owner or role with `can_manage_space` may edit/delete a channel. Any current group member may edit/delete the group, matching Section 4.6 of the course specification. Destructive actions require UI confirmation.

FR-07 - Media

Messages accept up to five files, each at most 10 MiB, in the supported image, video, audio, or general-document categories. Names and paths are sanitized, and attachment downloads require current space membership. A channel role may disable media while retaining text permission.

FR-08 - Roles and access restrictions

Channel roles are database records with editable names and capabilities: send messages, send media, manage topics, manage members, delete messages, manage roles, and manage the space. The owner implicitly has every capability. The service rejects assignment or creation that would grant a manager a capability the manager does not possess.

FR-09 - Message search

Search is case-insensitive, text-only, and scoped to the selected ChatSpace and optional topic. A non-member receives no object/content information.

FR-10 - User profiles

Each user edits username, email, avatar, biography, and group-invite policy. Authenticated users can open a read-only modal for another profile.

FR-11 - Notifications

New-message and membership events create persistent notifications. Users can view unread/all items, mark one as read, or mark all as read. Persistence means a brief connection loss does not lose an event.

5. Bonus Requirements

B-01 - Live messages and notifications

After an HTTP write commits, the backend publishes an event to the relevant Redis-backed Channels group. Space consumers verify authentication and membership at connection time and again before each event. A separate authenticated per-user socket carries notification and membership events. The React client reconnects and refreshes durable state when needed, so no F5 or Ctrl+R is required.

B-02 - Scheduled messages

The API stores a future message as `PENDING` in UTC. Celery Beat starts a due scan every five seconds. A worker locks due rows, rechecks membership and message/media/topic permission, changes valid rows to `SENT`, creates notifications, commits, and then publishes live events. A status guard makes retries idempotent. Pending records in PostgreSQL survive browser, worker, or broker downtime; the sender does not need to be online at dispatch.

6. Security and Reliability

- same-origin session authentication for REST and WebSocket;
- CSRF validation for unsafe HTTP requests, including login/registration;
- Django password hashers and password validators;
- object-level membership and capability checks;
- WebSocket origin validation and repeated membership checks;
- randomized storage paths, type/extension/size checks, and protected file downloads;
- request throttles at Django REST Framework and Nginx boundaries;
- secrets and production cookie/HTTPS settings controlled by environment;
- plain-text rendering of message/profile content in React;
- database constraints for unique identities, memberships, direct pairs, roles, topics, nonces, and valid scheduling states;
- transaction-on-commit realtime publishing so clients never see rolled-back data;
- idempotent scheduled delivery with permission revalidation.

The detailed checklist is stored in `docs/phase2/security.md`.

7. Verification and Quality Assurance

The repository includes automated backend tests for:

- authentication, password hashing, duplicate identity, profile privacy, and CSRF;
- direct-chat uniqueness, groups, channels, membership, topics, and roles;
- the complete message edit/delete/media/search permission matrix;
- protected attachment access and cleanup;
- persistent notifications;
- anonymous/non-member WebSocket rejection and event delivery;
- due, future, cancelled, repeated, and revoked-permission scheduled tasks;
- liveness/readiness endpoints.

Frontend tests cover key authentication, state, and interaction flows. CI runs configuration and migration checks, Ruff, Pytest, ESLint, Vitest, and the production Vite build. Docker-level health and HTTP smoke scripts are also included.

8. Running the Product

```
cp .env.example .env
docker compose up --build -d
docker compose exec backend python manage.py seed_demo
```

Open `http://localhost:8080`. The README contains demo credentials and a recommended two-browser presentation flow.

Quality commands:

```
make test
make check
make smoke
```

9. Team Contribution Areas

- **Sina Mohammadi - Product Owner / Accounts:** acceptance criteria, registration, authentication, profile flows, and traceability.
- **Mohammad Ermia Ghaseri - Scrum Master / Spaces:** groups, channels, memberships, topics, and iteration records.
- **Mehrshad Valizadeh Arjmand - Data & Backend:** schema, migrations, messages, attachments, search, notifications, and demo data.
- **Amir Mohammad Shahrezaei - Infrastructure:** Docker, Nginx, ASGI, WebSockets/Redis, Celery/RabbitMQ, and scheduled delivery.
- **Amir Hossein Ghasemipour - UI:** React design system, three-column chat, dialogs, message/media/search/scheduling, and responsive behavior.
- **Nima Notghi - Architecture & QA:** permission model, integration review, tests, CI, architecture documentation, and final conformance report.

10. Repository and Project Board

Repository: `https://github.com/Sina-rokna/SD-EchoMessenger`

Project board: `https://github.com/users/Sina-rokna/projects/2/views/1`

The repository retains the original Phase 1 report, diagrams, and wireframes, alongside the final source, migrations, tests, deployment files, and Phase 2 documentation.